

kanban-kit

Selbst-hostbares Kanban-Board. Entwickelt mit KI. Geführt von einem Menschen.

Ich habe kanban-kit gebaut, um zu zeigen, was KI-gestützte Entwicklung leisten kann, wenn harte Leitplanken den Takt vorgeben. Jede Zeile ist durch Tests, Mutationstests und statische Analysen abgesichert. Wenn eine Regel verletzt wird, bricht der Build. Das Ergebnis ist kein Prototyp, sondern ein Produkt im täglichen Einsatz.

~68.400

Zeilen Code, geschrieben von der KI, verantwortet von mir

> 2.100

Tests, davon 256 Integrationstests gegen echtes Postgres und MinIO

100 %

Line- und Branch-Coverage, Backend und Frontend, als hartes Build-Gate

996 / 996

Mutationen getötet: 100 % Mutation Score (PIT)

CODEBASIS

Backend (Java)

Produktivcode	339 Klassen , ~16.100 Zeilen
Architektur	8 Fachmodule, hexagonal geschnitten
Testcode	~27.900 Zeilen , das 1,7-fache des Produktivcodes
Testläufe	1.205 (949 Unit/Slice, 256 Integration via Testcontainers)

Frontend (TypeScript/React)

Produktivcode	86 Dateien , ~11.100 Zeilen
Testcode	~13.400 Zeilen , auch hier mehr Test als Code
Tests	930 (Vitest und Testing Library)
Coverage	100 % Line und Branch (v8), Build-Gate

LEITPLANKEN

Qualität, gemessen

Mutationstests	996 von 996 Mutationen getötet (PIT)
SonarCloud	Quality Gate grün, 0 Bugs, 0 Vulnerabilities , 0 % Duplikate

Regeln, die den Build brechen

Statisch	ArchUnit, Checkstyle, PMD, SpotBugs, Error Prone, NullAway/JSpecify, Spotless
Prinzip	Leitplanken argumentieren nicht. Sie scheitern.

Wer KI unkontrolliert arbeiten lässt, produziert schneller schlechten Code. Wer gelernt hat, sie zu führen, arbeitet schneller und besser. kanban-kit ist mein Beleg dafür.